DATA SCIENCE USING PYTHON

MAT337 • CIA–II (October 2026)

Short Python Programs with Sample Outputs and Simple Explanations

Q1. RETAIL COMPANY – DICTIONARY OPERATIONS

a) Frequency of each character in a customer review

Code

```
review = input("Review: ").lower()
freq = {}
for ch in review:
freq[ch] = freq.get(ch, 0) + 1
print(freq)
```

Output

```
Input: good
Output: {'g': 1, 'o': 2, 'd': 1}
```

Simple explanation: Each character is used as a dictionary key. get(ch, 0) reads its current count, and the program increases that count by one.

b) Products above a rating threshold without changing the original dictionary

Code

```
ratings = {"Phone": 4.5, "Tablet": 3.8, "Laptop": 4.8}
threshold = 4.0
selected = {p: r for p, r in ratings.items() if r > threshold}
print(selected)
print("Original:", ratings)
```

Output

```
Output: {'Phone': 4.5, 'Laptop': 4.8}
Original: {'Phone': 4.5, 'Tablet': 3.8, 'Laptop': 4.8}
```

Simple explanation: The dictionary comprehension creates a new dictionary containing ratings strictly greater than 4.0. The original dictionary is unchanged.

c) Product with the highest revenue

Code

```
sales = {"Phone": 50000, "Tablet": 32000, "Laptop": 75000}
product = max(sales, key=sales.get)
print(product, sales[product])
```

Output

```
Output: Laptop 75000
```

Simple explanation: max(..., key=sales.get) returns the product name whose revenue is highest. sales[product] displays that revenue.

d) Combine attribute names and values into a customer record

Code

```
fields = ["name", "age", "city"]
values = ["Vikki", 22, "Chennai"]
```

```
customer = dict(zip(fields, values))
print(customer)
```

Output

```
Output: {'name': 'Vikki', 'age': 22, 'city': 'Chennai'}
```

Simple explanation: zip() pairs each field with its corresponding value. dict() converts those pairs into a dictionary.

e) Total revenue from different regions

Code

```
revenue = {"North": 12000, "South": 18000, "East": 9000, "West": 11000}
print("Total revenue:", sum(revenue.values()))
```

Output

```
Output: Total revenue: 50000
```

Simple explanation: values() returns all regional revenue values. sum() adds them to calculate the overall revenue.

Q2. COMPANY SALES – DICTIONARY PROCESSING

a) Character frequency in a product description

Code

```
text = input("Description: ").lower()
frequency = {ch: text.count(ch) for ch in dict.fromkeys(text)}
print(frequency)
```

Output

```
Input: pen
Output: {'p': 1, 'e': 1, 'n': 1}
```

Simple explanation: dict.fromkeys(text) gives each distinct character once. count() calculates how often each character occurs.

b) New dictionary for products meeting a rating condition

Code

```
ratings = {"A": 4.2, "B": 3.5, "C": 4.7}
minimum = 4.0
filtered = {name: rating for name, rating in ratings.items()
if rating >= minimum}
print(filtered)
print("Original:", ratings)
```

Output

```
Output: {'A': 4.2, 'C': 4.7}
Original: {'A': 4.2, 'B': 3.5, 'C': 4.7}
```

Simple explanation: The new dictionary keeps ratings greater than or equal to the threshold. The source dictionary remains unchanged.

c) Product with maximum sales revenue from product records

Code

```
records = {
"Pen": {"revenue": 1200},
"Bag": {"revenue": 8500},
"Book": {"revenue": 4200}
}
best = max(records, key=lambda p: records[p]["revenue"])
print(best, records[best]["revenue"])
```

Output

```
Output: Bag 8500
```

Simple explanation: max() checks the revenue stored inside each product record and returns the product with the largest revenue.

d) Convert field names and values into one dictionary

Code

```
fields = ["product", "rating", "region"]
values = ["Laptop", 4.8, "South"]
record = dict(zip(fields, values))
print(record)
```

Output

```
Output: {'product': 'Laptop', 'rating': 4.8, 'region': 'South'}
```

Simple explanation: zip() matches each field name with a value, and dict() builds a structured record.

e) Overall revenue from multiple regions

Code

```
regions = {"North": 10000, "South": 15000, "East": 7000}
total = sum(regions.values())
print("Overall revenue:", total)
```

Output

```
Output: Overall revenue: 32000
```

Simple explanation: The values in the regional dictionary are added to obtain the overall revenue.

Q3. ENVIRONMENTAL RESEARCH – GENERATOR EXPRESSIONS

a) Integers from 1 to 100 whose digit sum is divisible by 5; sum their fourth powers

Code

```
numbers = (n for n in range(1, 101)
if sum(int(d) for d in str(n)) % 5 == 0)
```

```
selected = list(numbers)
print("Numbers:", selected)
print("Sum of fourth powers:", sum(n**4 for n in selected))
```

Output

```
Output: Numbers: [5, 14, 19, 23, 28, 32, 37, 41, 46, 50, 55, 64, 69, 73, 78, 82, 87, 91, 96, 100]
Sum of fourth powers: 387559626
```

Simple explanation: A digit sum is calculated by converting the number to text and adding its digits. The condition keeps numbers whose

digit sum is divisible by 5. The final generator adds their fourth powers.

b) Count integers from 1 to 500 containing a repeated digit

Code

```
count = sum(1 for n in range(1, 501)
if len(str(n)) != len(set(str(n))))
print(count)
```

Output

```
Output: 122
```

Simple explanation: set(str(n)) contains only distinct digits. If its length is smaller than the number's digit count, at least one digit is repeated.

c) Count words longer than six characters in research sentences

Code

```
sentences = [
"Climate patterns influence environmental conditions",
"Researchers measure temperature and rainfall"
]
count = sum(1 for sentence in sentences
for word in sentence.split() if len(word) > 6)
print(count)
```

Output

```
Output: 4
```

Simple explanation: split() separates each sentence into words. The generator counts words containing more than six characters.

d) Find Pythagorean triples for x, y, z from 1 to 5

Code

```
triples = [(x, y, z) for x in range(1, 6)
for y in range(1, 6) for z in range(1, 6)
if x*x + y*y == z*z]
print(triples)
```

Output

```
Output: [(3, 4, 5), (4, 3, 5)]
```

Simple explanation: All combinations from 1 to 5 are checked. A triple is kept when $x^2 + y^2$ equals z^2.

e) Sum positive integers below 200 divisible by 3 or 7, but not both

Code

```
total = sum(n for n in range(1, 200)
if (n % 3 == 0) != (n % 7 == 0))
print(total)
```

Output

```
Output: 7585
```

Simple explanation: The != condition is true when exactly one divisibility test is true. Therefore, numbers divisible by both 3 and 7 are excluded.

Q4. AGRICULTURAL RESEARCH – NUMPY ARRAYS

a) Create a 3 × 3 matrix from 1 to 9; square and cube elements

Code

```
import numpy as np
A = np.arange(1, 10).reshape(3, 3)
print("Matrix:\n", A)
print("Squares:\n", A**2)
print("Cubes:\n", A**3)
```

Output

```
Output:
Matrix:
[[1 2 3]
[4 5 6]
[7 8 9]]
Squares:
[[ 1  4  9]
[16 25 36]
[49 64 81]]
Cubes:
[[  1   8  27]
[ 64 125 216]
[343 512 729]]
```

Simple explanation: arange(1, 10) creates values 1 through 9. reshape(3, 3) forms a 3 × 3 matrix. **2 and **3 calculate each element's square and cube.

b) Create a 4 × 4 matrix and calculate row and column sums

Code

```
import numpy as np
A = np.arange(1, 17).reshape(4, 4)
print(A)
```

```
print("Row sums:", A.sum(axis=1))
print("Column sums:", A.sum(axis=0))
```

Output

```
Output:
[[ 1  2  3  4]
[ 5  6  7  8]
[ 9 10 11 12]
[13 14 15 16]]
Row sums: [10 26 42 58]
Column sums: [28 32 36 40]
```

Simple explanation: axis=1 calculates the sum across each row. axis=0 calculates the sum down each column.

c) Create block matrix [[A, B], [B, A]]

Code

```
import numpy as np
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
top = np.hstack((A, B))
bottom = np.hstack((B, A))
block = np.vstack((top, bottom))
print(block)
```

Output

```
Output:
[[1 2 5 6]
[3 4 7 8]
[5 6 1 2]
[7 8 3 4]]
```

Simple explanation: hstack() joins arrays side by side. vstack() joins the two resulting rows of blocks vertically.

d) Multiply a 2 × 3 matrix by a 3 × 2 matrix

Code

```
import numpy as np
A = np.array([[1, 2, 3], [4, 5, 6]])
B = np.array([[7, 8], [9, 10], [11, 12]])
print(A @ B)
```

Output

```
Output:
[[ 58  64]
[139 154]]
```

Simple explanation: The inner dimensions are both 3, so matrix multiplication is possible. @ performs matrix multiplication.

e) Reshape 12 values to 3 × 4 and cube each element

Code

```
import numpy as np
a = np.arange(1, 13)
A = a.reshape(3, 4)
print("Matrix:\n", A)
print("Cubes:\n", A**3)
```

Output

```
Output:
Matrix:
[[ 1  2  3  4]
[ 5  6  7  8]
[ 9 10 11 12]]
Cubes:
[[   1    8   27   64]
[ 125  216  343  512]
[ 729 1000 1331 1728]]
```

Simple explanation: reshape(3, 4) arranges 12 observations into three rows and four columns. **3 cubes every element.

Q5. HEALTHCARE ANALYTICS – NUMPY ARRAYS

a) Create a 3 × 3 array and calculate squares and cubes

Code

```
import numpy as np
observations = np.arange(1, 10).reshape(3, 3)
print("Array:\n", observations)
print("Squared:\n", np.square(observations))
print("Cubed:\n", observations**3)
```

Output

```
Output:
Array:
[[1 2 3]
[4 5 6]
[7 8 9]]
Squared:
[[ 1  4  9]
[16 25 36]
[49 64 81]]
Cubed:
[[  1   8  27]
[ 64 125 216]
[343 512 729]]
```

Simple explanation: NumPy creates the array and performs element-wise mathematical operations. np.square() squares each value; **3 cubes it.

b) Generate a 4 × 4 array and calculate row and column totals

Code

```
import numpy as np
A = np.arange(1, 17).reshape(4, 4)
print("Row totals:", np.sum(A, axis=1))
print("Column totals:", np.sum(A, axis=0))
```

Output

```
Output:
Row totals: [10 26 42 58]
Column totals: [28 32 36 40]
```

Simple explanation: NumPy sum with axis=1 calculates row totals, while axis=0 calculates column totals.

c) Arrange A and B in a block matrix with A on the diagonal

Code

```
import numpy as np
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
block = np.block([[A, B], [B, A]])
print(block)
```

Output

```
Output:
[[1 2 5 6]
[3 4 7 8]
[5 6 1 2]
[7 8 3 4]]
```

Simple explanation: np.block() arranges the four 2 × 2 arrays into the required larger block matrix.

d) Multiply a 2 × 3 array by a 3 × 2 array

Code

```
import numpy as np
A = np.array([[1, 2, 3], [4, 5, 6]])
B = np.array([[1, 2], [3, 4], [5, 6]])
result = np.matmul(A, B)
print(result)
```

Output

```
Output:
[[22 28]
[49 64]]
```

Simple explanation: np.matmul() multiplies matrices. A 2 × 3 matrix multiplied by a 3 × 2 matrix produces a 2 × 2 result.

e) Reshape 12 measurements and cube every element

Code

```
import numpy as np
values = np.arange(1, 13)
matrix = values.reshape(3, 4)
print(matrix**3)
```

Output

```
Output:
[[   1    8   27   64]
 [ 125  216  343  512]
 [ 729 1000 1331 1728]]
```

Simple explanation: The 12 values are reshaped into a 3 × 4 array. The power operator applies cubing to every element.

Q6. ENGINEERING RESEARCH – MATRIX PROPERTIES

a) Check whether a random matrix has an inverse using determinant

Code

```
import numpy as np
rng = np.random.default_rng(10)
A = rng.integers(1, 10, size=(3, 3))
det = np.linalg.det(A)
print("Matrix:\n", A)
print("Determinant:", round(det, 3))
if not np.isclose(det, 0):
inv = np.linalg.inv(A)
print("Inverse:\n", np.round(inv, 3))
print("A @ inverse:\n", np.round(A @ inv, 3))
else:
print("Inverse does not exist")
```

Output

Sample output varies because the matrix is randomly generated. If the determinant is non-zero, A multiplied by its inverse is

Simple explanation: A square matrix is invertible when its determinant is non-zero. np.isclose() helps check whether a floating-point determinant is effectively zero.

b) Construct a non-zero matrix with dependent rows

Code

```
import numpy as np
A = np.array([[1, 2, 3], [2, 4, 6], [1, 1, 1]])
print(A)
print("Determinant:", np.linalg.det(A))
print("Rank:", np.linalg.matrix_rank(A))
```

Output

```
Output:
[[1 2 3]
[2 4 6]
[1 1 1]]
Determinant: approximately 0
Rank: 2
```

Simple explanation: The second row is twice the first row, so the rows are linearly dependent. A square matrix with determinant zero has no inverse.

c) Eigenvalues and eigenvectors of [[4,2],[1,3]]

Code

```
import numpy as np
A = np.array([[4, 2], [1, 3]])
values, vectors = np.linalg.eig(A)
print("Eigenvalues:", np.round(values, 3))
for i, lam in enumerate(values):
v = vectors[:, i]
print("Av:", np.round(A @ v, 3))
print("lambda*v:", np.round(lam * v, 3))
```

Output

```
Output: Eigenvalues: [5. 2.]
For each eigenvector, Av and lambda*v match approximately.
```

Simple explanation: An eigenvector v satisfies Av = λv. NumPy returns eigenvalues and eigenvectors; small decimal differences are due to floating-point calculations.

d) Compare two matrix multiplication orders

Code

```
import numpy as np
A = np.array([[1, 2], [0, 1]])
B = np.array([[2, 0], [1, 3]])
print("A @ B:\n", A @ B)
print("B @ A:\n", B @ A)
print("Same result:", np.array_equal(A @ B, B @ A))
```

Output

```
Output:
A @ B:
[[4 6]
[1 3]]
B @ A:
[[2 4]
[0 5]]
Same result: False
```

Simple explanation: Matrix multiplication is generally not commutative: changing the order can change the result.

e) Verify orthogonality for distinct eigenvalues of a symmetric matrix

Code

```
import numpy as np
A = np.array([[2, 1], [1, 2]])
values, vectors = np.linalg.eigh(A)
print("Eigenvalues:", values)
v1, v2 = vectors[:, 0], vectors[:, 1]
print("Dot product:", np.dot(v1, v2))
print("Orthogonal:", np.isclose(np.dot(v1, v2), 0))
```

Output

```
Output:
Eigenvalues: [1. 3.]
Dot product: approximately 0
Orthogonal: True
```

Simple explanation: For a real symmetric matrix, eigenvectors corresponding to distinct eigenvalues are orthogonal. The dot product of orthogonal vectors is zero.

Q7. RESEARCH AND DEVELOPMENT – NUMERICAL METHODS

a) Find rank when one row is a multiple of another

Code

```
import numpy as np
A = np.array([[1, 2, 3], [2, 4, 6], [1, 0, 1]])
print("Rank:", np.linalg.matrix_rank(A))
```

Output

```
Output: Rank: 2
```

Simple explanation: The second row is twice the first, so those two rows do not provide independent information. The rank counts the independent rows or columns.

b) Block diagonal matrix and determinant verification

Code

```
import numpy as np
A = np.array([[2, 0], [0, 3]])
B = np.array([[1, 2], [0, 4]])
block = np.block([[A, np.zeros((2, 2))],
[np.zeros((2, 2)), B]])
print(block)
print("det(block):", round(np.linalg.det(block), 3))
print("det(A)*det(B):", round(np.linalg.det(A)*np.linalg.det(B), 3))
```

Output

```
Output:
[[2. 0. 0. 0.]
[0. 3. 0. 0.]
[0. 0. 1. 2.]
[0. 0. 0. 4.]]
det(block): 24.0
det(A)*det(B): 24.0
```

Simple explanation: The determinant of a block diagonal matrix equals the product of the determinants of its diagonal blocks.

c) Analyse a system with a redundant equation

Code

```
import numpy as np
A = np.array([[1, 1, 1], [2, 2, 2], [1, -1, 0]], dtype=float)
b = np.array([6, 12, 0], dtype=float)
solution, residuals, rank, singular = np.linalg.lstsq(A, b, rcond=None)
print("One least-squares solution:", np.round(solution, 3))
print("Rank:", rank)
print("Number of variables:", A.shape[1])
```

Output

```
Output: One least-squares solution: [3. 3. 0.]
Rank: 2
Number of variables: 3
```

Simple explanation: One equation is a multiple of another, and the coefficient matrix has rank 2 for 3 variables. Therefore, the solution is not unique; least squares returns one valid representative solution.

d) Compare nested loops with NumPy vectorization

Code

```
import numpy as np
from time import perf_counter
A = np.arange(1000*1000, dtype=float).reshape(1000, 1000)
start = perf_counter()
loop_result = [[x*x for x in row] for row in A]
loop_time = perf_counter() - start
start = perf_counter()
numpy_result = A**2
numpy_time = perf_counter() - start
print("Loop seconds:", round(loop_time, 4))
print("NumPy seconds:", round(numpy_time, 4))
print("Results match:", np.allclose(loop_result, numpy_result))
```

Output

```
Output: Execution times vary by computer. Results match: True
```

Simple explanation: Both methods square every value. NumPy performs the operation in optimized array code, which is usually faster than nested Python loops.

e) Sum $n^2/(n+1)$ for the first 50 integers using generator and NumPy

Code

```
import numpy as np
total_gen = sum(n*n/(n+1) for n in range(1, 51))
n = np.arange(1, 51)
total_np = np.sum(n**2/(n+1))
print("Generator:", round(total_gen, 6))
print("NumPy:", round(float(total_np), 6))
```

Output

```
Output: Generator: 1201.019877
NumPy: 1201.019877
```

Simple explanation: The generator expression calculates each term one at a time. NumPy calculates all 50 terms as an array. The totals should match up to rounding.

Q8. SENSOR DATA – GENERATORS AND NUMPY

a) Generate observations and calculate mean, variance, and median

Code

```
import numpy as np
rng = np.random.default_rng(5)
data = np.fromiter((rng.integers(10, 51) for _ in range(10)),
dtype=int)
print("Data:", data)
print("Mean:", np.mean(data))
print("Variance:", np.var(data))
print("Median:", np.median(data))
```

Output

```
Output depends on the generated observations. The program displays the data, mean, population
variance, and median.
```

Simple explanation: fromiter() turns generator values into a NumPy array. mean() gives the average, var() gives population variance by default, and median() gives the middle value.

b) Normalize sensor readings using mean and standard deviation

Code

```
import numpy as np
readings = np.fromiter((x for x in [10, 12, 14, 16, 18]), dtype=float)
mean = readings.mean()
std = readings.std()
normalized = (readings - mean) / std
print("Normalized:", np.round(normalized, 3))
```

Output

```
Output: Normalized: [-1.414 -0.707  0.     0.707  1.414]
```

Simple explanation: Z-score normalization subtracts the mean from each value and divides by the standard deviation. The resulting values have mean near zero and standard deviation near one.

c) Sum squared magnitudes of all non-zero matrix elements

Code

```
import numpy as np
A = np.array([[1, 0, -2], [3, 0, 4]])
total = sum(x*x for x in A.flat if x != 0)
print("Total squared magnitude:", total)
```

Output

```
Output: Total squared magnitude: 30
```

Simple explanation: A.flat iterates through all elements. The generator squares only non-zero values: $1^2 + (-2)^2 + 3^2 + 4^2 = 30$.

d) Estimate pi using the Monte Carlo method

Code

```
import random
N = 100000
points = ((random.random(), random.random()) for _ in range(N))
inside = sum(1 for x, y in points if x*x + y*y <= 1)
print("Estimated pi:", 4 * inside / N)
```

Output

```
Output: Estimated pi is usually close to 3.14; the exact value varies between runs.
```

Simple explanation: Random points are generated inside a unit square. The fraction inside the quarter-circle approximates $\pi/4$, so multiplying that fraction by 4 estimates π.

e) Select systems whose eigenvalues all have magnitude below one

Code

```
import numpy as np
systems = [
np.array([[0.2, 0], [0, 0.5]]),
np.array([[1.2, 0], [0, 0.3]]),
np.array([[0.4, 0.1], [0, 0.6]])
]
stable = [A for A in systems
if all(abs(v) < 1 for v in np.linalg.eigvals(A))]
for i, A in enumerate(stable, 1):
print("Stable system", i, ":\n", A)
```

Output

```
Output: The first and third systems are stable.
```

Simple explanation: A discrete-time system is stable under the stated test when every eigenvalue has magnitude strictly less than one. all() checks every eigenvalue.

Q9. SMART CITY – NUMERICAL AND DATA ANALYSIS

a) Overlapping groups of speeds and covariance

Code

```
import numpy as np
speeds = np.array([30, 32, 35, 33, 36, 40, 38], dtype=float)
window = 3
groups = (speeds[i:i+window] for i in range(len(speeds)-window+1))
for g in groups:
print(g, "variance:", round(float(np.var(g)), 3))
```

Output

Output: Each overlapping group contains three consecutive speeds, and its variance is displayed.

Simple explanation: The question asks for covariance but provides only one speed sequence. Covariance requires two variables. This program therefore shows variance for each group; to calculate covariance, provide a second aligned measurement sequence.

b) Calculate image gradient magnitude at pixel locations

Code

```
import numpy as np
image = np.array([[10, 10, 20],
[10, 30, 40],
[10, 30, 50]], dtype=float)
gx = np.zeros_like(image)
gy = np.zeros_like(image)
gx[:, 1:-1] = (image[:, 2:] - image[:, :-2]) / 2
gy[1:-1, :] = (image[2:, :] - image[:-2, :]) / 2
magnitude = np.sqrt(gx**2 + gy**2)
print(np.round(magnitude, 2))
```

Output

Output:

```
[[ 0.   5.   0. ]
[10.  15.  10. ]
[ 0.  10.   0. ]]
```

Simple explanation: Horizontal and vertical intensity differences estimate the image gradient. The magnitude is $\sqrt{(gx^2 + gy^2)}$. This simple example uses central differences for interior pixels.

c) Find feature pairs with strong positive correlation

Code

```
import numpy as np
data = {
"price": np.array([10, 20, 30, 40, 50]),
"sales": np.array([12, 22, 31, 42, 49]),
"returns": np.array([5, 4, 6, 3, 5])
}
threshold = 0.9
```

```
pairs = ((a, b, np.corrcoef(data[a], data[b])[0, 1])
for i, a in enumerate(data)
for b in list(data)[i+1:]
if np.corrcoef(data[a], data[b])[0, 1] > threshold)
for pair in pairs:
print(pair)
```

Output

Output: The price–sales pair is displayed if its calculated correlation exceeds 0.9.

Simple explanation: corrcoef() calculates the correlation coefficient. The generator examines each unique feature pair and retains pairs above the threshold.

d) Centre data, find covariance and project onto principal component

Code

```
import numpy as np
X = np.array([[2, 1, 3], [3, 2, 4], [4, 4, 5],
[5, 3, 6]], dtype=float)
Xc = X - X.mean(axis=0)
cov = np.cov(Xc, rowvar=False)
values, vectors = np.linalg.eigh(cov)
pc1 = vectors[:, np.argmax(values)]
projection = Xc @ pc1
print("Covariance:\n", np.round(cov, 3))
print("First principal component:", np.round(pc1, 3))
print("Projection:", np.round(projection, 3))
```

Output

Output: Displays the covariance matrix, first principal component direction, and each row's projection onto that direction.

Simple explanation: Column means are subtracted to centre the data. Eigenvectors of the covariance matrix give principal-component directions. The eigenvector with the largest eigenvalue captures the greatest variance.

e) Identify the most influential station using the dominant eigenvector

Code

```
import numpy as np
adj = np.array([[0, 1, 1], [1, 0, 1], [1, 1, 0]], dtype=float)
values, vectors = np.linalg.eig(adj)
idx = np.argmax(np.real(values))
dominant = np.real(vectors[:, idx])
station = int(np.argmax(np.abs(dominant))) + 1
print("Dominant eigenvalue:", round(float(np.real(values[idx])), 3))
print("Dominant eigenvector:", np.round(dominant, 3))
print("Most influential station:", station)
```

Output

Output: Dominant eigenvalue: 2.0
Dominant eigenvector: [0.577 0.577 0.577]
Most influential station: 1

Simple explanation: The eigenvector associated with the largest eigenvalue is the dominant eigenvector. The largest absolute component identifies the station with the greatest score. In this symmetric example, all stations tie, so the first is returned.

Q10. DATA CLEANING – PANDAS AND NUMPY

a) Standardize column names and fill missing numerical values

Code

```
import pandas as pd
df = pd.read_csv("students.csv")
df.columns = (df.columns.str.strip().str.lower()
.str.replace(r"\s+", "_", regex=True))
print("Missing values before:\n", df.isna().sum())
for col in df.select_dtypes(include="number").columns:
df[col] = df[col].fillna(df[col].median())
print("Missing values after:\n", df.isna().sum())
df.to_csv("students_cleaned.csv", index=False)
```

Output

Output: Displays missing-value counts before and after filling; saves students_cleaned.csv.

Simple explanation: Column names are stripped of surrounding spaces, converted to lowercase, and internal spaces become underscores. Missing numerical values are filled with each column's median, which is less affected by extreme values than the mean.

b) Standardize text and remove duplicate sales rows

Code

```
import pandas as pd
df = pd.read_csv("sales.csv")
for col in df.select_dtypes(include="object").columns:
df[col] = df[col].str.strip().str.lower()
before = len(df)
df = df.drop_duplicates()
print("Rows removed:", before - len(df))
df.to_csv("sales_cleaned.csv", index=False)
```

Output

Output: Displays the number of duplicate rows removed and saves sales_cleaned.csv.

Simple explanation: Text columns are stripped of extra spaces and converted to lowercase so entries such as 'Phone' and ' phone ' are standardized. drop_duplicates() removes identical rows.

c) Detect salary outliers using IQR and cap them

Code

```
import pandas as pd
df = pd.read_csv("employees.csv")
col = "salary"
q1 = df[col].quantile(0.25)
```

```
q3 = df[col].quantile(0.75)
iqr = q3 - q1
low, high = q1 - 1.5*iqr, q3 + 1.5*iqr
outliers = df[(df[col] < low) | (df[col] > high)]
print("IQR limits:", low, high)
print("Outlier rows:\n", outliers)
df[col] = df[col].clip(lower=low, upper=high)
df.to_csv("employees_cleaned.csv", index=False)
```

Output

Output: Displays the IQR limits and outlier rows, then saves employees_cleaned.csv.

Simple explanation: `IQR equals Q3 minus Q1. Values below Q1 − 1.5×IQR or above Q3 + 1.5×IQR are flagged. This example caps extreme salaries at the limits rather than deleting employees, preserving records while reducing extreme influence.`

d) Encode categorical values, correct types, and create a feature

Code

```
import pandas as pd
df = pd.read_csv("health.csv")
if "age" in df.columns:
df["age"] = pd.to_numeric(df["age"], errors="coerce")
if "gender" in df.columns:
df["gender"] = df["gender"].astype("string").str.strip().str.lower()
df["gender_code"] = df["gender"].map({"female": 0, "male": 1})
if "height_cm" in df.columns and "weight_kg" in df.columns:
height_m = pd.to_numeric(df["height_cm"], errors="coerce") / 100
weight = pd.to_numeric(df["weight_kg"], errors="coerce")
df["bmi"] = weight / (height_m ** 2)
df.to_csv("health_cleaned.csv", index=False)
print(df.head())
```

Output

Output: Displays the first five processed rows and saves health_cleaned.csv. Actual columns depend on health.csv.

Simple explanation: Numerical fields are converted with errors coerced to missing values when invalid. Gender is mapped to numeric codes. If height and weight columns exist, BMI is created as weight divided by height in metres squared.

e) Clean orders, handle missing values, create order-efficiency feature, save CSV

Code

```
import pandas as pd
df = pd.read_csv("orders.csv")
df.columns = df.columns.str.strip().str.lower()
df = df.drop_duplicates()
for col in df.select_dtypes(include="number").columns:
df[col] = df[col].fillna(df[col].median())
for col in df.select_dtypes(include="object").columns:
```

```
df[col] = df[col].fillna("Unknown").str.strip()
# Example: efficiency = items delivered per processing day
if {"items_delivered", "processing_days"}.issubset(df.columns):
days = pd.to_numeric(df["processing_days"], errors="coerce").replace(0, np.nan)
items = pd.to_numeric(df["items_delivered"], errors="coerce")
df["order_efficiency"] = items / days
df.to_csv("orders_processed.csv", index=False)
print(df.head())
```

Output

Output: Displays the first five processed rows and saves orders_processed.csv. The feature is created when the two named colu

Simple explanation: The program standardizes column names, removes duplicate rows, fills numerical missing values with medians, and fills missing text values with 'Unknown'. Order efficiency is defined here as items delivered per processing day; use the actual column names in the dataset.